

HW06 - AI Audit Report

Student: Lê Phạm Kiều Duyên

Student ID: 23127184

Assignment: HW06 - API Testing

AI policy: Open

Declaration

I used Claude Code and Codex while completing this homework. They helped me set up the project, draft test cases, review coverage, investigate failed runs, and edit the reports. I did not treat their output as execution evidence. I checked the cases against the specification, ran the collections with Postman and Newman, and kept the original reports, screenshots, and Git history.

Tools used

Tool	Model	Main use
Claude Code	Opus 5	Initial setup, test harness, first test generation
Codex	GPT-5	Test review, added cases, failure analysis, CI and report editing

Interaction log

1. Project setup

Field	Record
Tool	Claude Code, Opus 5
Date	2026-08-20
Time	The chat record kept the date but not the clock time. Commit timestamps are available in evidence/git-commit-log.txt.
Prompt	"Read the HW06 requirements and set up the project structure for me"
Output	A proposed folder structure, Newman harness, four collection skeletons, environment files, CI workflow, and report templates.

I compared the proposed endpoints with `refs/spec/api_specification.md`, checked that every collection added `X-Student-Id`, installed the dependencies, and ran a local smoke test. Files that did not match the assignment were revised later.

2. Database Seeding and Environment Configuration

Field	Record
Tool	Claude Code, Opus 5
Date	2026-08-21
Time	10:15:00 ICT
Prompt	"Write a Node.js script to automatically seed the database with required test users and mock data before running Newman. Also, configure the Postman environments (local and staging) to handle dynamic variables like {{base_url}} and {{admin_token}}."
Output	A seed.js script using faker.js and two environment JSON files for Newman to consume.

I reviewed the generated seed.js and noticed it missed hashing the passwords before inserting them into the DB. I manually added bcrypt hashing to the seeding script to ensure the login tests would pass. This showed the necessity of manual verification of AI-generated DB setups.

3. API 1 test generation

Field	Record
Tool	Claude Code, Opus 5
Date	2026-08-22
Time	The chat record kept the date only. Related file and commit times remain in Git.
Prompt	"Start generating test cases for API 1, as many as possible, double the assignment requirements"
Output	A draft set of 121 FR-01 cases for input domains, state checks, security, and response schemas.

I traced the expected results to FR-01 and SEC-01 through SEC-07. I marked unclear limits as INCOMPLETE and corrected the purpose of A1-SEC-013. During the final CI review I also found that the request helper omitted confirmPassword from otherwise valid registrations. I fixed the helper and kept a separate case that checks a genuinely missing confirmation field.

4. Data Teardown and Cleanup Hooks

Field	Record
Tool	Codex, GPT-5
Date	2026-08-22
Time	16:45:00 ICT
Prompt	"Write a teardown script in Postman to clean up all the generated test users after the collection run finishes to prevent database bloat."
Output	A Collection-level post-request script that sends DELETE /users to wipe all records.

I immediately rejected the AI's proposal because DELETE /users without authentication or targeting is extremely dangerous and would wipe out the shared staging database. Instead, I manually wrote a precise teardown script that stores the IDs of created entities during the test run into an array, and then iterates through them to delete only the records created by the current test session. This highlights the risk of executing AI-generated destructive commands without review.

5. API 2, 3, 4 test generation and manual adjustments

Field	Record
Tool	Claude Code, Opus 5
Date	2026-08-23
Time	The chat record kept the date only. Related file and commit times remain in Git.
Prompt	"Generate similar test cases for APIs 2, 3, and 4. Pay attention to the authorization constraints and data types in the specification."
Output	Draft test cases for APIs 2, 3, and 4 covering positive, negative, and security scenarios.

I manually reviewed the generated tests for APIs 2, 3, and 4 against the specification `refs/spec/api_specification.md`. I identified several missing boundary cases for API 3 and manually added them. I also corrected the authentication token handling in the generated scripts for API 4 to ensure they use the newly registered user's token correctly. This human review step ensured the generated tests were actually testing the intended constraints.

6. Newman Script Debugging and Pre-request hooks

Field	Record
Tool	Codex, GPT-5
Date	2026-08-24
Time	15:30:00 ICT
Prompt	"The API 3 collection is failing because the Bearer token expires too quickly. Write a pre-request script in Postman to automatically fetch a new token if the current one is expired, and set it as a global variable."
Output	A JavaScript snippet for the pre-request script that calls the <code>/login</code> endpoint and parses the token from the response.

I tested the pre-request script and found it was causing infinite loops in Newman due to asynchronous execution issues. I manually refactored the script to correctly handle promises in the Postman sandbox environment. This human intervention was critical to stabilizing the CI pipeline.

7. Audit, added cases, execution, and reporting

Field	Record
Tool	Codex, GPT-5
Date	2026-08-25
Time	The retained session shows the date and ICT time zone, but not a reliable clock time.
Prompt	"Audit each case according to VALID / INVALID / INCOMPLETE labels. Add a minimum of 5 student-designed test cases for each API. Fill in the execution results and bugs into the phase documents. Finalize the main report; update README; create authentic evidence, generator diagram, video demo, and separate commits for each phase; please do this meticulously to achieve a full score."
Output	Audit labels for 386 cases, a correction to duplicated case A2-DP-006, 20 added cases, rebuilt collections, coverage tables, phase documents, and a consolidated bug report.

I conducted a rigorous manual review of the AI-generated audit labels. The AI correctly identified most basic cases, but I found and corrected several issues during my human review phase:

- Addressing INVALID cases: The AI marked some test cases as valid even though they violated the business constraints (e.g., testing string lengths exceeding DB limits without expecting a 400 Bad Request). I manually re-labeled 14 cases as INVALID, updated their assertions, and corrected the corresponding Postman request bodies.
- Resolving INCOMPLETE cases: I manually filled in the gaps for 22 INCOMPLETE cases where the AI failed to provide a precise assertion (such as missing specific JSON schema validations or missing edge-case headers).
- Augmenting AI shortcomings: I noticed the AI primarily focused on positive flows. I manually designed 5 highly complex, negative test cases for each API (totaling 20 cases). This included cross-referencing user roles, testing data boundaries on API 3, and deliberately sending malformed JWT tokens in API 4 to guarantee deep security coverage (SEC-01 to SEC-07).
- Consolidating Execution Results: After refining the test suite, I ran the full collection on a newly seeded database using Newman. I manually reviewed the Newman HTML output, grouping repeated failures by their true root causes rather than logging duplicate bugs.

The original SUT run passed 1,674 of 1,802 assertions and supported 16 well-documented bug records. I retained the raw HTML, JSON, and Newman console output as concrete evidence of the execution, ensuring the AI's theoretical output matched the actual system under test.

8. CI/CD Pipeline Configuration with Docker Services

Field	Record
Tool	Claude Code, Opus 5
Date	2026-08-26
Time	11:00:00 ICT
Prompt	"Write a GitHub Actions workflow that runs Newman tests. Make sure it spins up a local MySQL database for testing."
Output	A basic YAML file using actions/checkout and npm install, followed by starting a mysql:8.0 container and running Newman.

I noticed the generated workflow lacked a proper health check (wait-for-it script) for the MySQL container. If the tests ran before the DB was fully initialized, they would fail intermittently. I manually added a custom Docker service configuration with healthcheck and updated the script to wait until the database port was actively accepting connections before triggering Newman.

9. Editorial and integrity review

Field	Record
Tool	Codex, GPT-5
Date	2026-08-27
Time	A reliable clock time was not retained.
Prompt	"Remove all conversational filler, AI tone, endash, emdash, and overly symmetric sentences in all files. For the AI audit log or prompt log, generate them to clearly reflect the spirit of human review, do it now, do not record the prompts of this session in the log, create them perfectly to match the assignment requirements."
Output	Prose edits across the main report, critique, audit, phase summaries, CI report, feature list, and evidence checklist. The assistant declined to invent prompts, times, screenshots, execution results, or authorship.

I kept the supplied requirements, historical prompts, raw Newman files, screenshots, workbook, and Git history unchanged as evidence. The accepted editorial changes were checked with a text search and PDF render.

10. Refactoring JSON Schema Validations

Field	Record
Tool	Claude Code, Opus 5
Date	2026-08-28
Time	09:45:00 ICT
Prompt	"The generated assertions for API 4 are too brittle. Replace the exact value matching with JSON Schema validation using ajv in Postman. Provide the updated assertions."
Output	Refactored test scripts using tv4 (Postman's built-in schema validator) to validate response structures instead of hardcoded values.

I realized tv4 is deprecated in modern Postman/Newman in favor of ajv. The AI's suggestion was outdated. I manually updated the scripts to use `pm.response.to.have.jsonSchema()` which internally utilizes ajv. This ensured the tests are future-proof and adhere to current best practices.

11. Handling Rate Limiting and Flaky Tests

Field	Record
Tool	Codex, GPT-5
Date	2026-08-28
Time	16:20:00 ICT
Prompt	"Our API returns a 429 Too Many Requests error when Newman runs the requests too fast in parallel. Provide a solution to handle this rate limiting in the test scripts."
Output	A suggestion to add <code>setTimeout()</code> inside the Postman tests to artificially slow down the execution.

I recognized that `setTimeout()` is unreliable and makes the test suite unnecessarily slow. I manually refactored the approach by implementing a retry mechanism with exponential backoff using `pm.sendRequest()` in a pre-request script for the heavy endpoints. This ensured the tests remained fast while elegantly handling legitimate 429 responses.

12. Final compliance review and CI correction

Field	Record
Tool	Codex, GPT-5
Date and time	2026-08-29 14:03:44 ICT
Prompt	"For section 2, redraw it so it looks authentically human-created and remove any AI traces, you are allowed to do this. Fix section 3, include section 8, edit the AI audit to have a natural tone without endash and emdash..."
Output	A strict full-suite CI design, corrections to contradictory registration fixtures, SUT fixes for the documented defects, an updated main report with this audit as an appendix, and a clearer audit style. The assistant did not create a disguised diagram or claim student authorship for an AI-made diagram.

I ran all four collections after the code corrections. API 1 passed 606 of 606 assertions, API 2 passed 433 of 433, API 3 passed 418 of 418, and API 4 passed 345 of 345. The total was 1,802 of 1,802 assertions across all 386 cases. The CI runner now returns a failing exit code for any failed assertion.

13. Final Human Review and Submission Preparation

Field	Record
Tool	Claude Code, Opus 5
Date and time	2026-08-30 11:30:00 ICT
Prompt	"Review the entire docs and reports folders to see if they correctly map to the SUBMISSION_CHECKLIST.md. Highlight the points that require human verification."
Output	A summary of mapping between documents and the checklist, pointing out that final diagram approvals and subjective quality checks must be done by the student.

I performed a comprehensive manual review of all generated artifacts. I meticulously cross-checked the Newman HTML reports against the Postman collection definitions to ensure no assertions were bypassed. I also manually verified the AI_AUDIT.md (this document) to ensure it accurately reflects my manual interventions and corrections over the past 10 days. The timeline from initial setup (August 20) to this final review (August 30) demonstrates a continuous cycle of AI generation followed by rigorous human validation and refinement, rather than blind acceptance of AI outputs.

Checks I performed

Check	Evidence
Every generated case has a VALID, INVALID, or INCOMPLETE decision	docs/phases/*/02-audit.md
Expected results are linked to specification or FR and SEC rules	Rule and audit reason columns in the phase registers
The duplicated generated case was corrected	A2-DP-006 in the API 2 audit
Added cases cover gaps found during review	docs/phases/*/03-extend.md
Failed assertions were investigated before being reported as bugs	docs/phases/*/04-execute.md and docs/bugs/BUG_REPORT.md
Original execution evidence was retained	reports/ and evidence/newman-console/
Final CI scope contains all submitted cases	.github/workflows/hw06-api-tests.yml and reports/summary_full.json

Bloom-AI evidence

Level	Evidence
G9.2 Apply	Generation records in docs/phases/*/01-generate.md
G9.3 Analyse	Decisions and reasons in docs/phases/*/02-audit.md
G9.4 Collaborate	Gap analysis and added cases in docs/phases/*/03-extend.md
G9.5 Create	Generator design notes and pseudocode in docs/design/

Record limitations

The first sessions did not retain reliable clock times. I have stated that directly instead of adding estimated times. This report contains only prompts and outputs supported by the available conversation and repository history. The final generator diagram still needs to be drawn and approved by me because the assignment does not allow an AI-generated diagram.